

Lógica y Representación 1

Funciones - Introducción

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

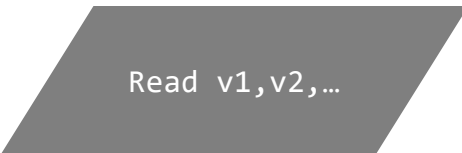
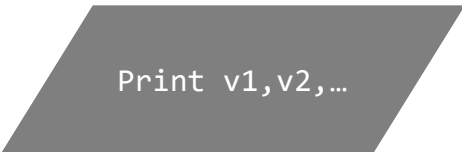


Outputs



Tabla resumen

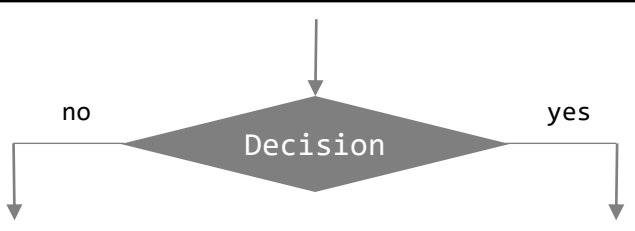
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Tabla resumen

La siguiente tabla, agrega la nueva el nuevo bloque que permite implementar toma de decisiones en los programas.

Diagrama de bloques	Pseudocodigo	Python
	Si (condicion) Entonces instrucciones verdad Sino instrucciones falsa Fin_Si	<pre>if (condicion): instrucciones verdad else: instrucciones falso</pre>

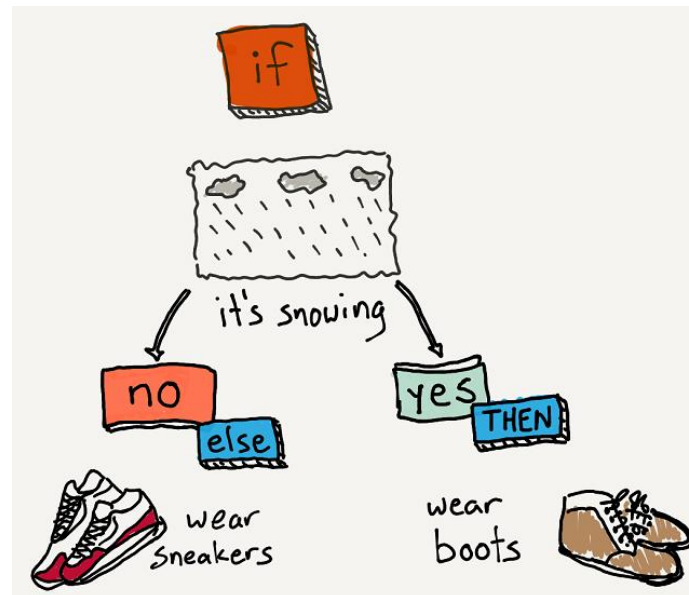


Imagen tomada de CS Intro ([link](#))

Casos en los problemas de decisión lógica

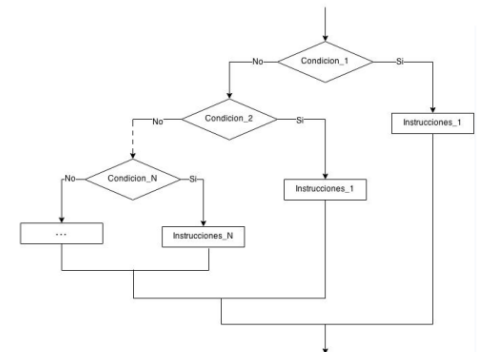
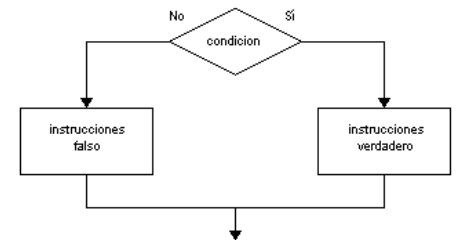
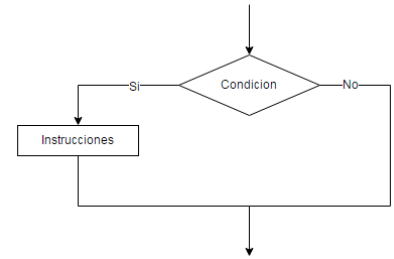


**Alternativas en los
problemas de decisión
lógica**

Alternativa simple

Alternativa doble

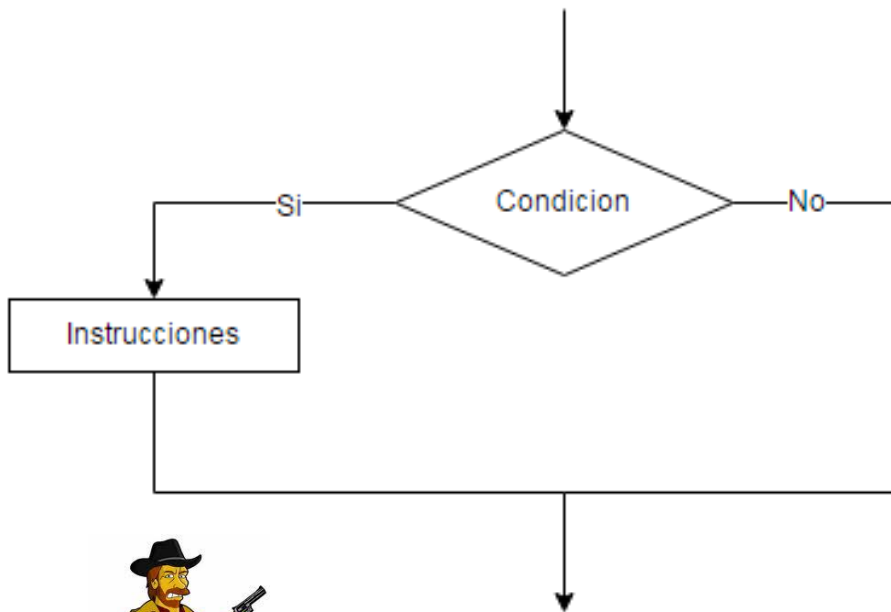
Alternativa múltiple



Alternativa simple

En los problemas de este tipo la condición **falsa** no posee instrucciones.

Diagrama de flujo



Pseudocódigo

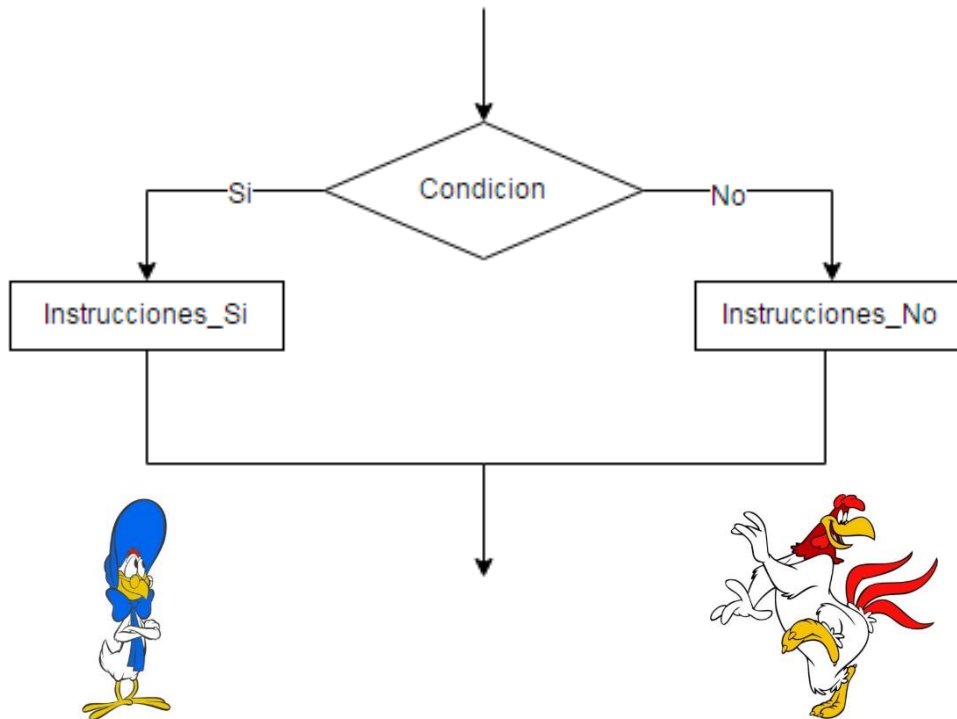
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera
```

Alternativa doble

En los problemas de este tipo, ambos casos (**Falso** y **Verdadero**) poseen instrucciones asociadas.



Pseudocódigo

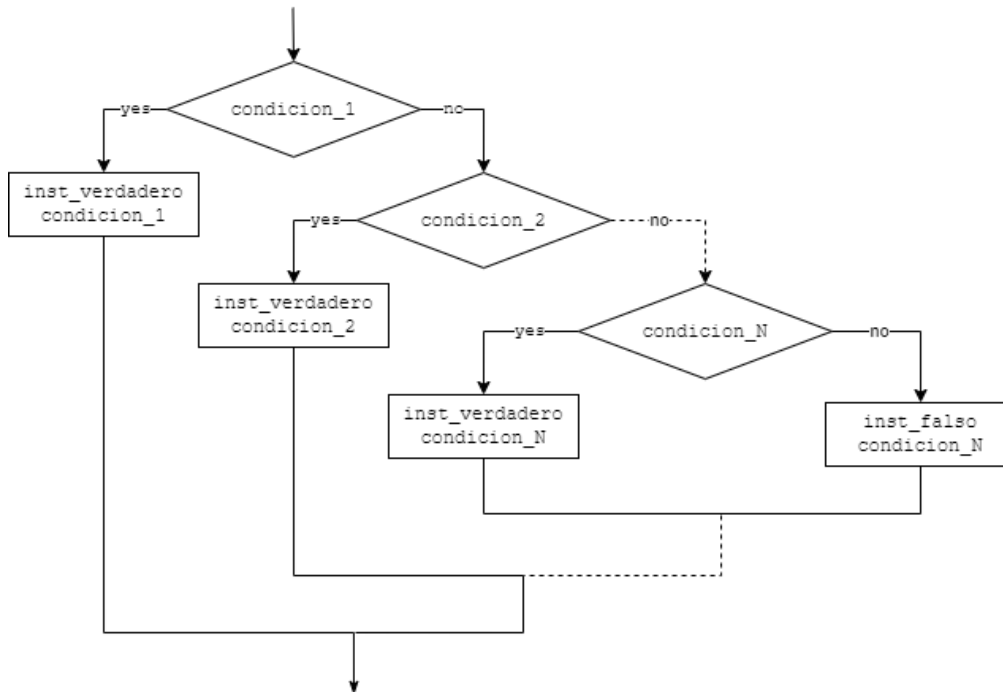
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Sino  
    instrucciones condición falsa  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera  
else:  
    instrucciones condición falsa
```


Alternativa múltiple – Mismo criterio

Diagrama de flujo



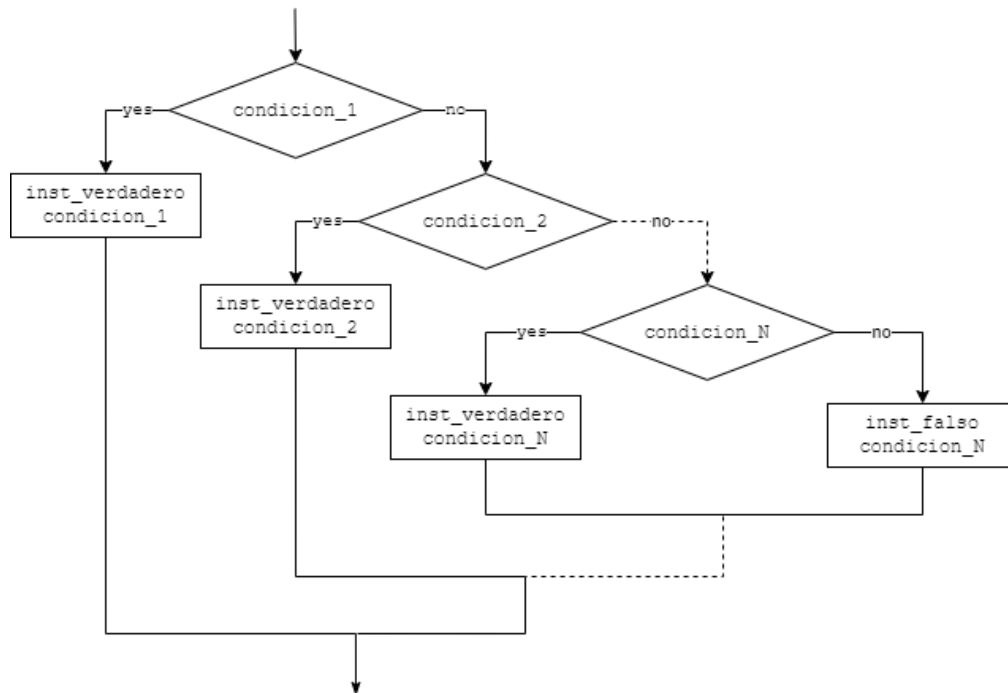
Pseudocódigo

```
Si (condicion_1) Entonces  
    condicion_1 verdad  
Sino  
    Si (condicion_2) Entonces  
        condicion_2 verdad  
    Sino  
        Si (condicion_3) Entonces  
            condicion_3 verdad  
        Sino  
            ...  
            Si (condicion_n) Entonces  
                condicion_n verdad  
            Sino  
                condicion_n falso  
            Fin_Si  
            ...  
        Fin_Si  
    Fin_Si  
Fin_Si
```

Alternativa múltiple – Mismo criterio

python

Diagrama de flujo

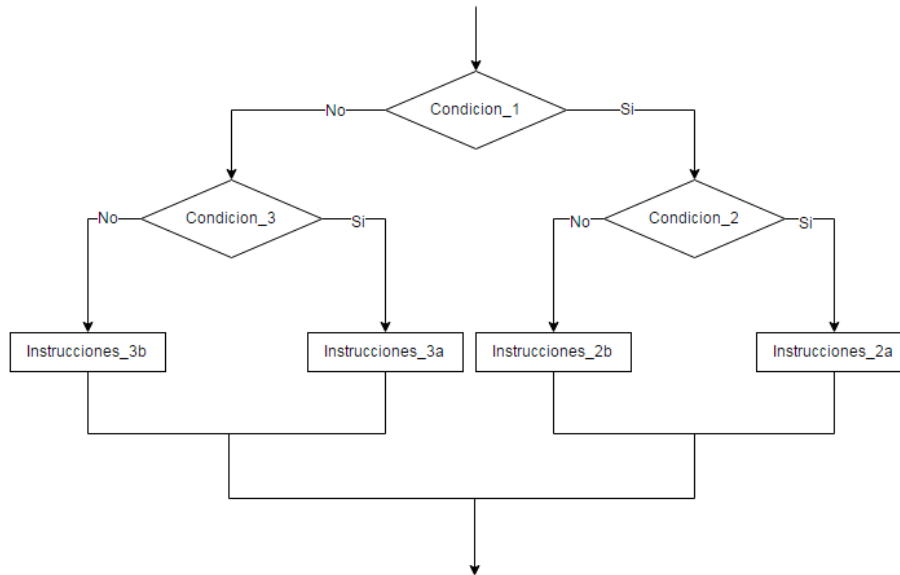


```
if (condicion_1):  
    condicion_1 verdad  
else:  
    if (condicion_2):  
        condicion_2 verdad  
    else:  
        if (condicion_3):  
            condicion_3 verdad  
        else:  
            ...  
            if (condicion_n):  
                condicion_n verdad  
            else:  
                condicion_n falso
```

```
if (condicion_1):  
    condicion_1 verdad  
elif (condicion_2):  
    condicion_2 verdad  
elif (condicion_3):  
    condicion_3 verdad  
...  
elif (condicion_n):  
    condicion_n verdad  
else:  
    condicion_n falso
```

Alternativa múltiple – Diferente criterio

Diagrama de flujo

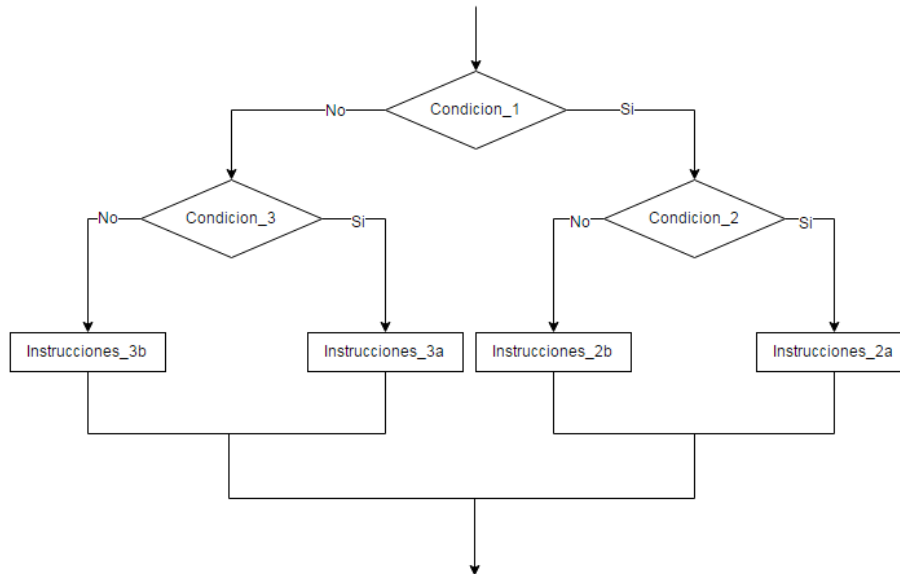


Pseudocódigo

```
Si (condicion_1) Entonces
    Si (condicion_3) Entonces
        condicion_3a verdad
    Sino
        condicion_3b
    Fin_Si
Sino
    Si (condicion_2) Entonces
        condicion_2a
    Sino
        condicion_2b
    Fin_Si
Fin_Si
```

Alternativa múltiple – Diferente criterio

Diagrama de flujo

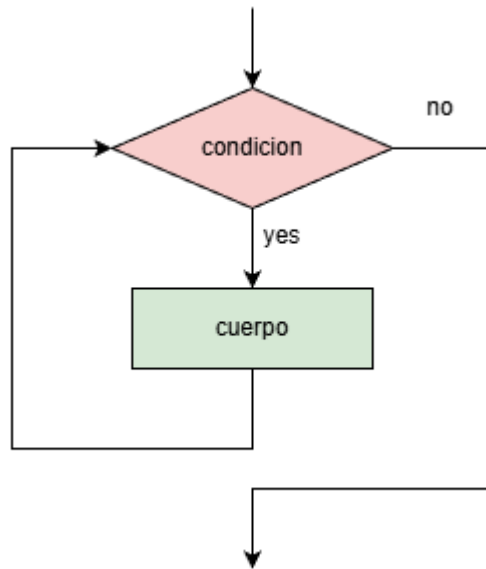


Python

```
if (condicion_1):  
    if (condicion_3):  
        condicion_3a verdad  
    else:  
        condicion_3b  
else:  
    if (condicion_2):  
        condicion_2a  
    else:  
        condicion_2b
```

Ciclo mientras

Diagrama de flujo



Pseudocódigo

```
Mientras (condicion) Haga  
    instrucciones cuerpo  
Fin_Mientras
```

Python

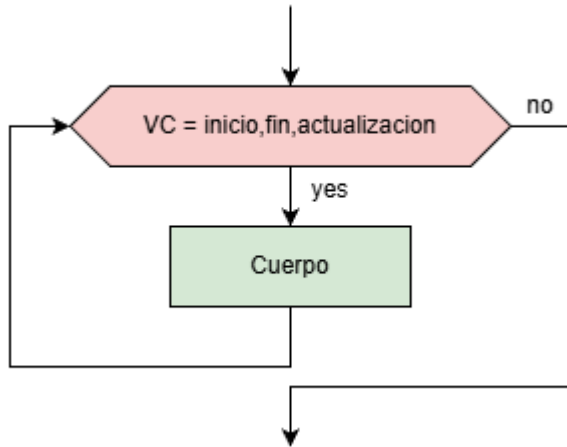
```
while (condicion):  
    instrucciones cuerpo
```

Observaciones importantes:

- Note que aunque la inicialización es importante para el ciclo, es previa a este (por eso no aparece).
- Dentro del cuerpo del ciclo están las instrucciones propias de lo que en este se debe realizar y la actualización de la variable de condición

Ciclo Para

Diagrama de flujo



Pseudocódigo

```
Para (vc = inicio, fin, actualizacion) Haga  
    instrucciones cuerpo  
Fin_Para
```

Python

```
for vc in range(inicio, fin + 1, actualizacion):  
    instrucciones cuerpo
```

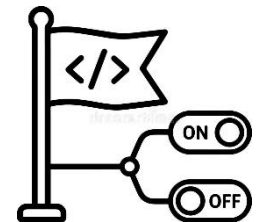
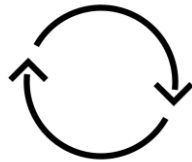
Observaciones importantes:

- A diferencia del ciclo **Mientras**, el ciclo **Para** se caracteriza por que la inicialización, la condición de control (o parada) y la actualización de la variable de control (**VC**) se definen en la cabecera del ciclo.
- Lo anterior hace que la actualización sea un proceso automático, lo cual lo hace la opción mas apropiada para ciclos donde el numero de iteraciones es conocido.
- En python la implementación del **Para**, es diferente a la de lenguajes como C++ y Java, por lo que hay que tener en cuenta unos detalles adicionales que veremos luego.

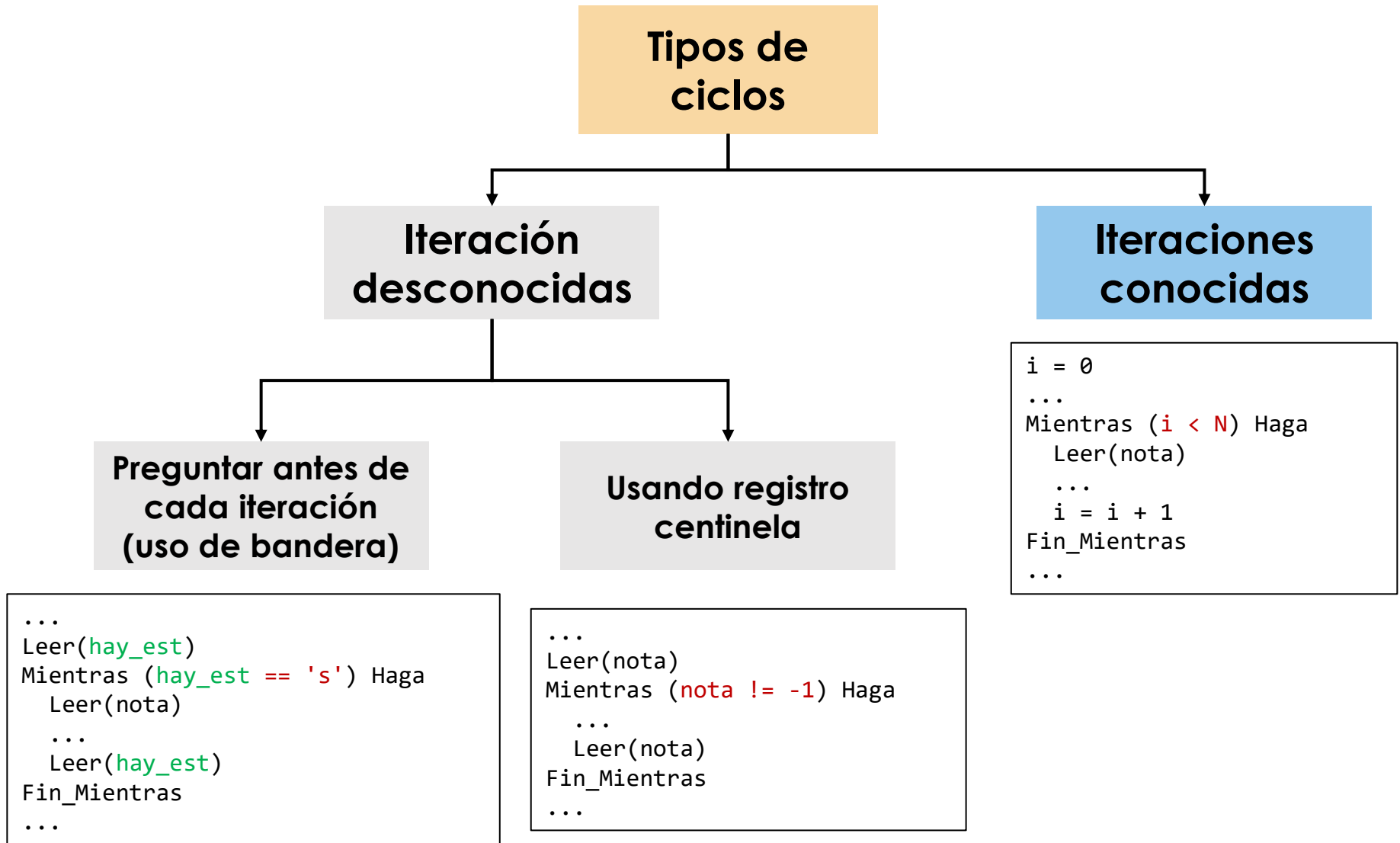
Tipos de variables útiles en los ciclos

Todo ciclo suele apoyarse en algunos tipos de variables que cumplen roles específicos:

- **Variable de control:** se evalúa en la condición del ciclo para decidir si continúa o termina
- **Contador:** lleva la cuenta de repeticiones; se actualiza en una cantidad constante.
- **Acumulador:** acumula el resultado de una operación que se repite; se actualiza en un valor variable.
- **Bandera:** variable de dos valores excluyentes usada para controlar el ciclo o recordar un estado dentro de él.
- **Centinela:** valor especial, fuera del rango válido de los datos, que señala el fin de la entrada.

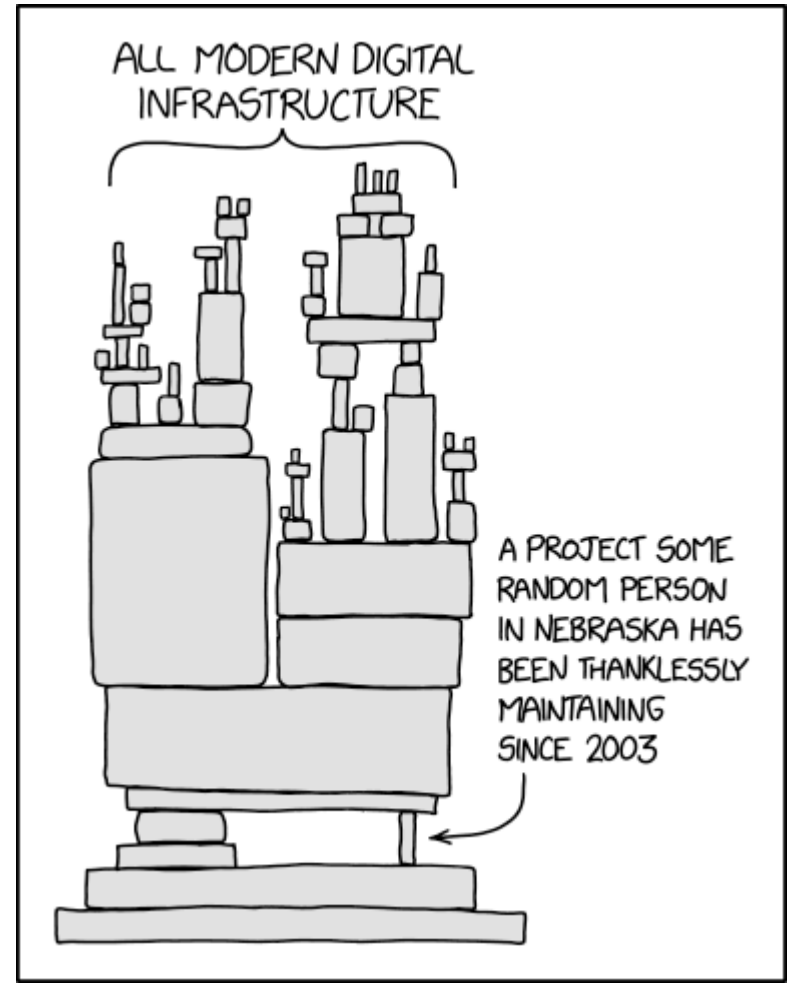


Recapitulando lo visto



Introducción a las funciones

```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
              // guaranteed to be random.  
}
```



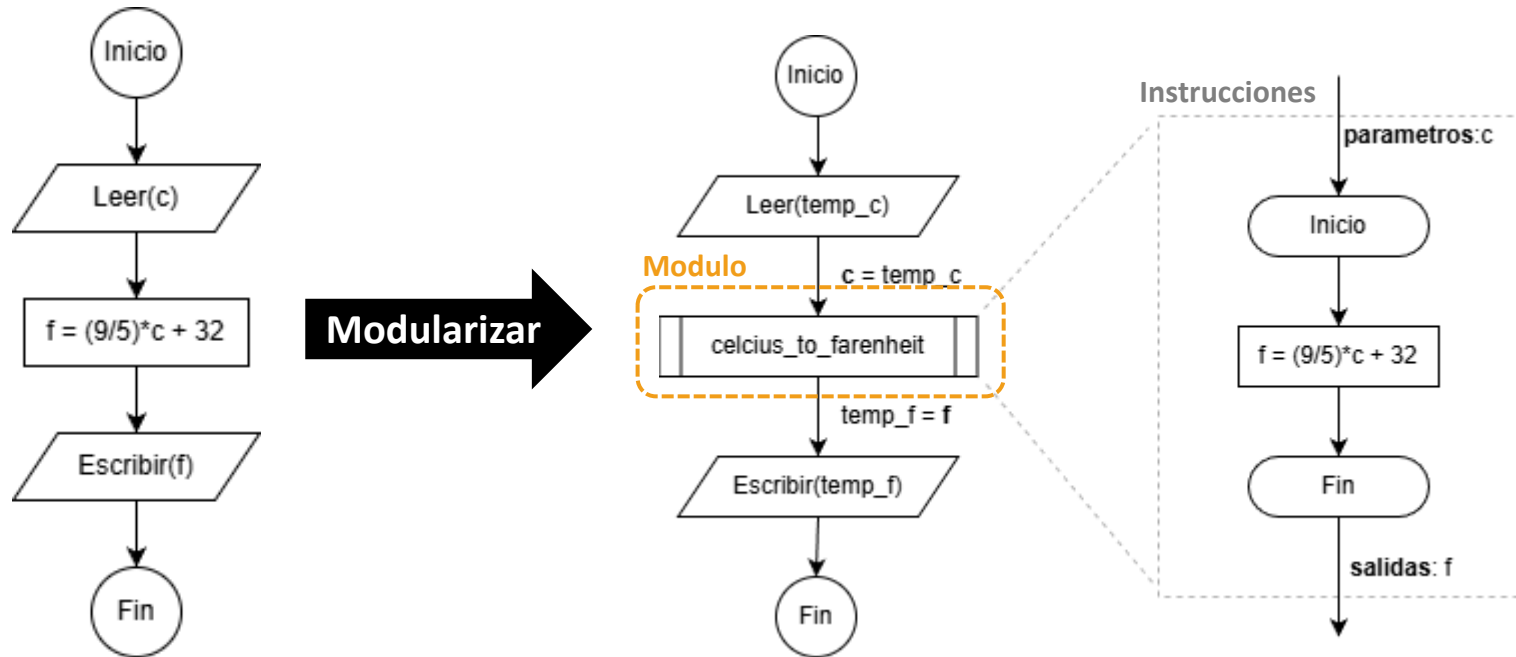
Módulos, subrutinas, procedimientos, funciones o métodos

Modulo:

- Porción de código que realiza una tarea específica.
- También se conoce como: subrutina, procedimiento, función o método.

Modularizar:

- Dividir un programa grande en módulos.

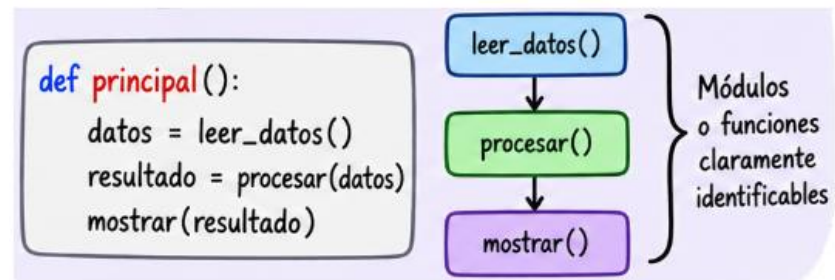
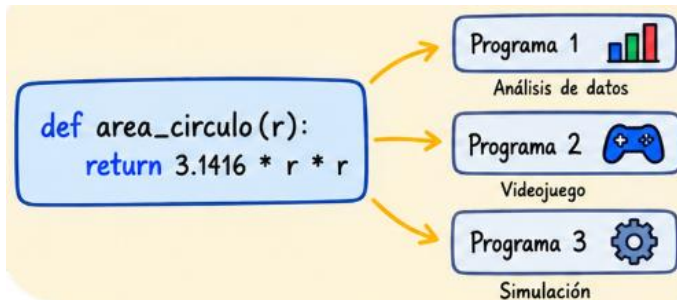
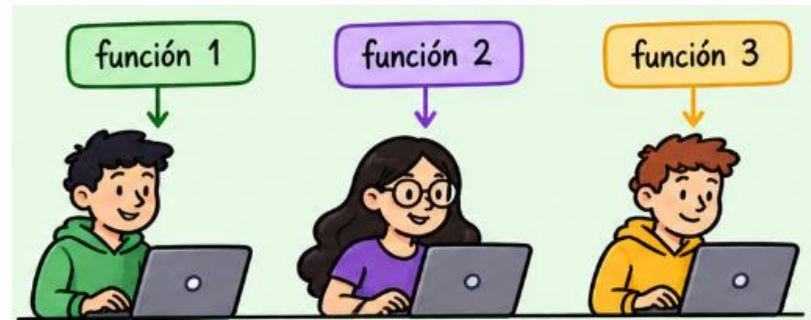
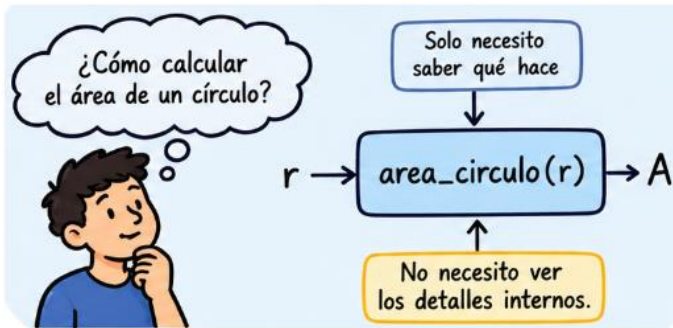


```
c = float(input("Ingrese los °C: "))  
f = (9.0/5)*c + 32  
print(c,"°C =",f, "°F")
```

[Link simulación](#)

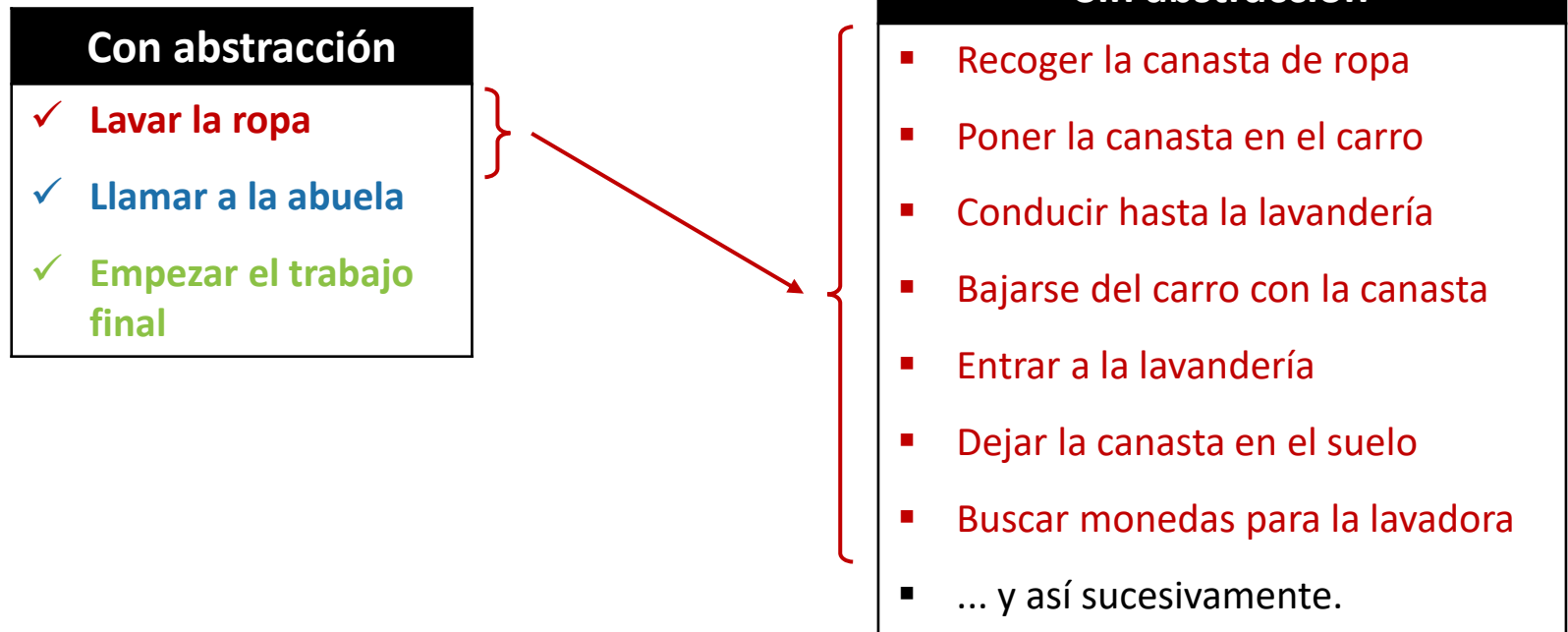
Ventajas de la modularización

- Proporciona abstracción.
- Permite el trabajo simultáneo de múltiples programadores.
- Permite el reuso de código.
- Facilita la identificación de estructuras mas fácilmente.



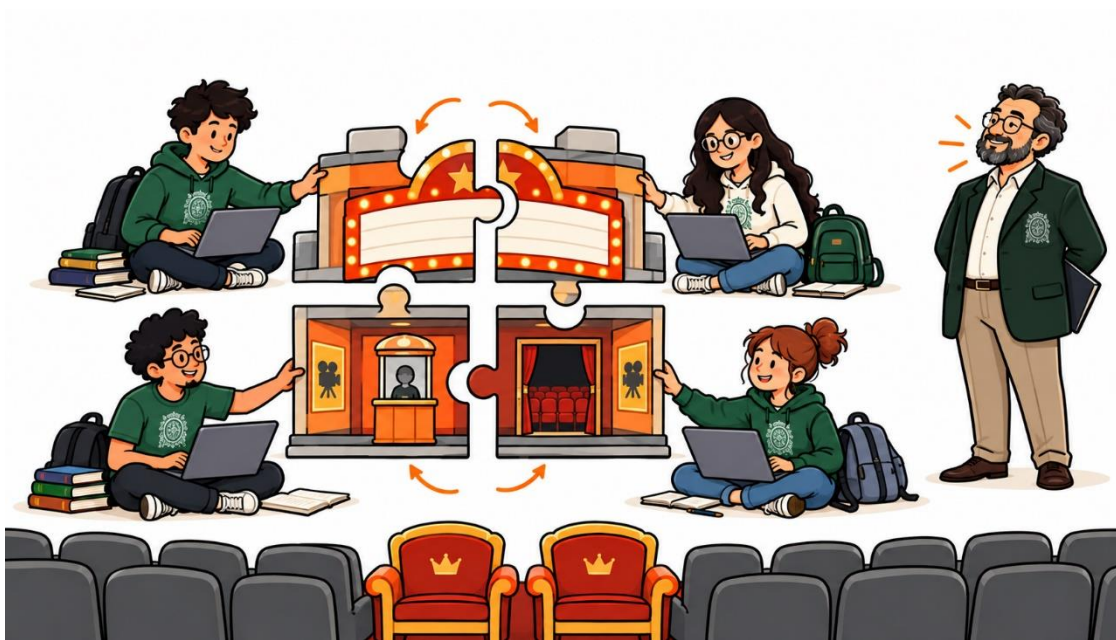
Abstracción

- Permite enfocarse en las propiedades importantes ignorando los detalles no esenciales.
- Evita detalles de bajo nivel y usa un enfoque de alto nivel
- Hace que tareas complejas parezcan simples.
- **Ejemplo:** Lista de tareas pendientes (To Do List)



Trabajo simultaneo

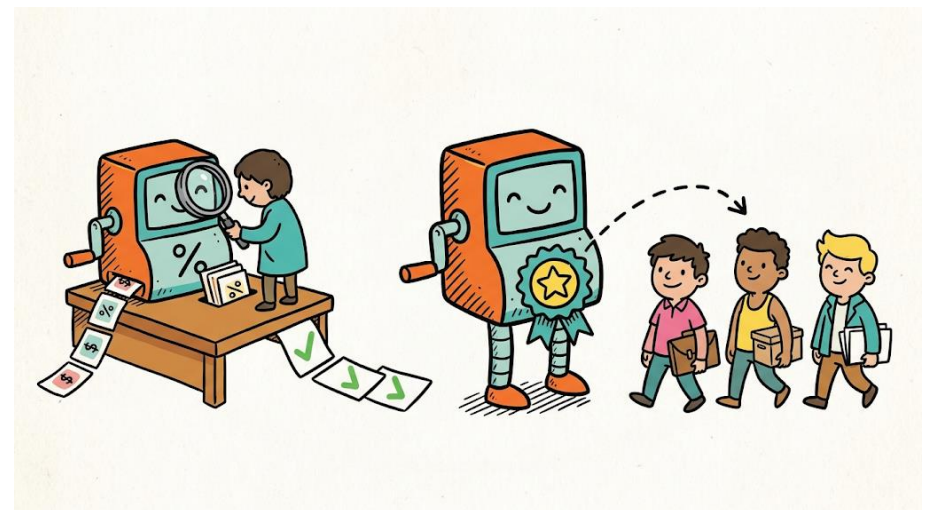
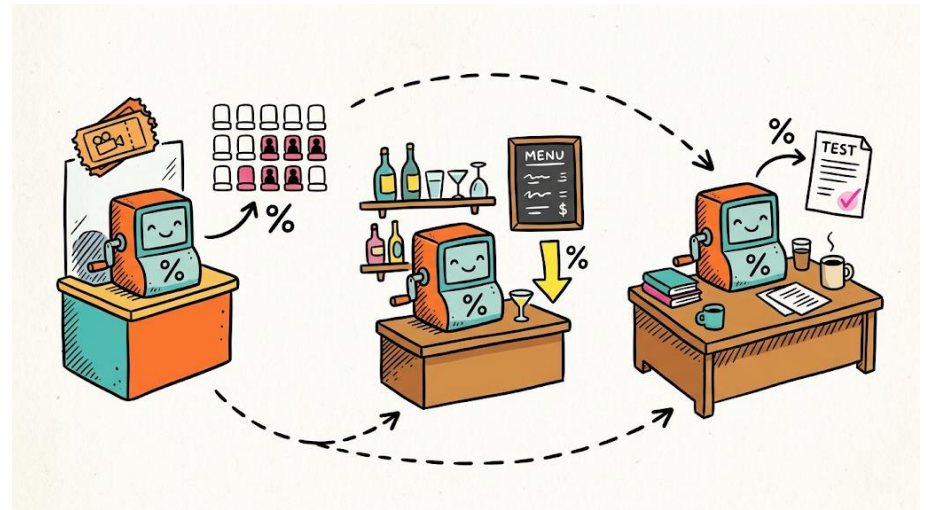
- Los proyectos de programación grandes pueden ser divididos en módulos.
- Los módulos pueden ser escritos por diferentes programadores.
- El tiempo de desarrollo se reduce significativamente.



Reutilización y confiabilidad

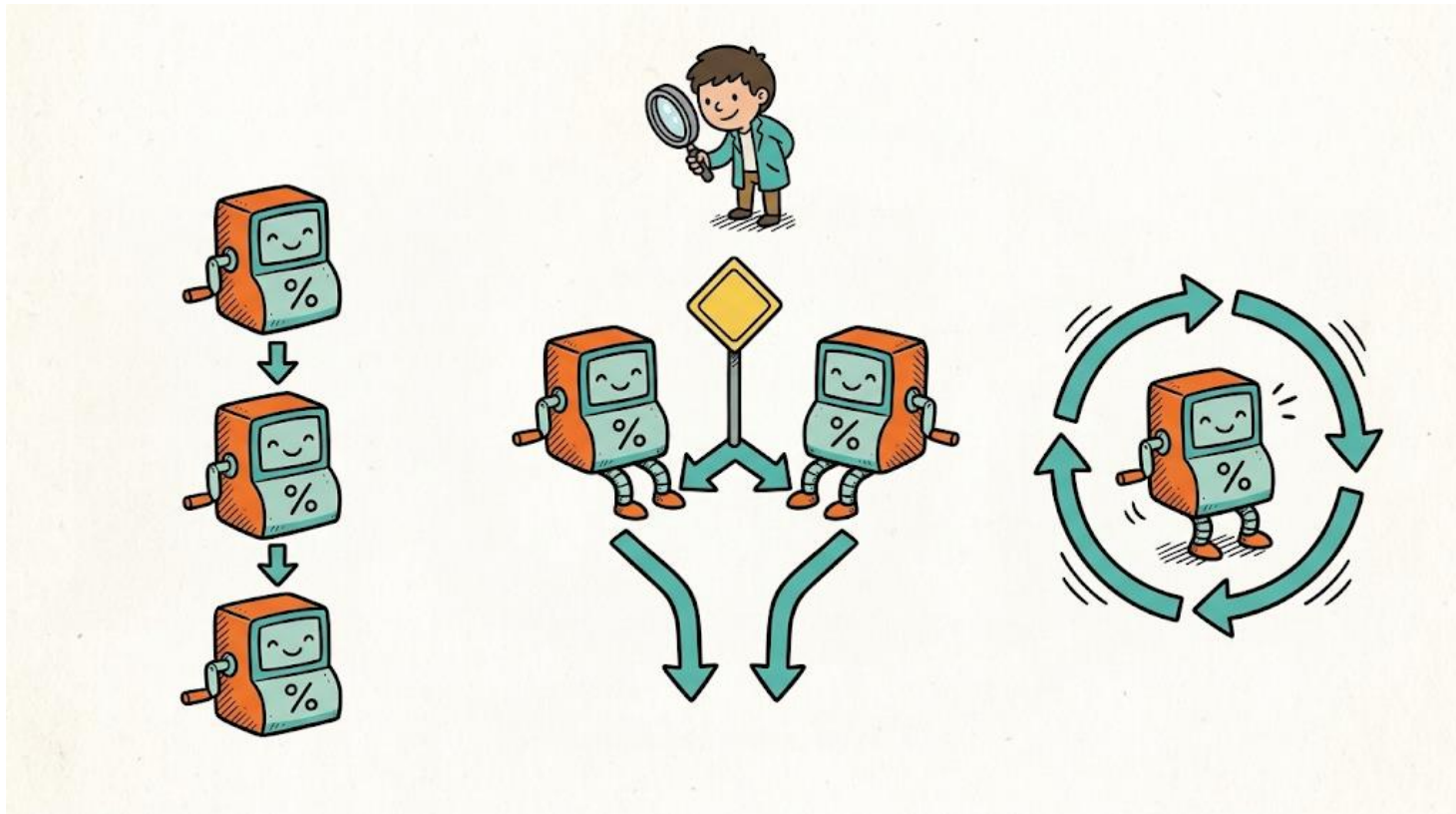
Al modularizar, es posible aprovechar código previamente creado para hacer nuevas cosas:

- **Reutilización:** capacidad de usar módulos en una variedad de aplicaciones.
- **Confiabilidad:** garantía de que un módulo ha sido probado y ha demostrado funcionar correctamente



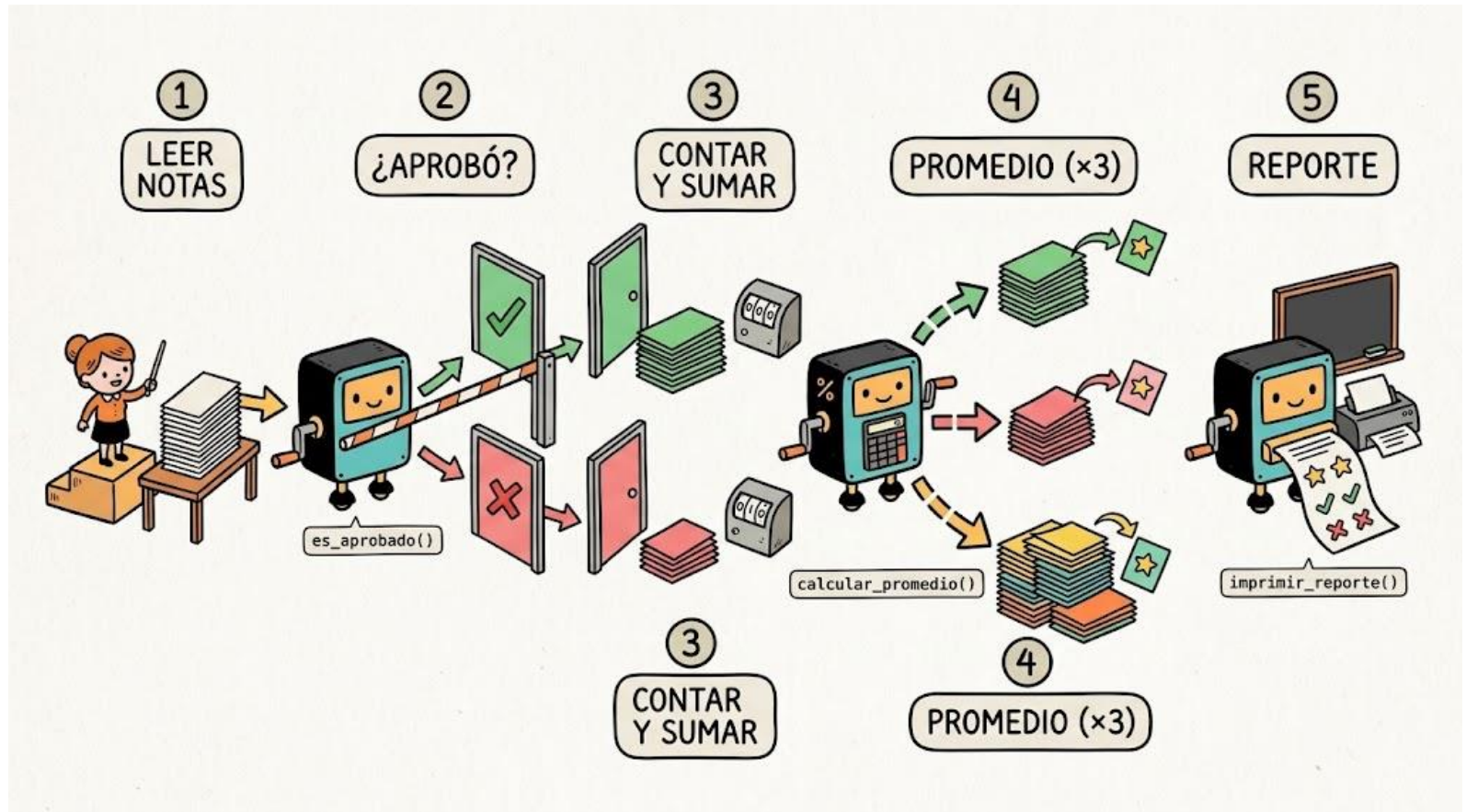
Facilitando la identificación de estructuras

- Agrupar varias tareas en módulos puede facilitar que los programadores principiantes:
 - Determinen si un programa es estructurado.
 - Identifiquen las estructuras de un programa.
- Los programadores experimentados modularizan por la abstracción, la facilidad para dividir el trabajo y la reutilización.



Modularizando un programa

- La mayoría de los programas contienen un **módulo principal**
 - Contiene la **lógica principal**
 - Accede a otros módulos o subrutinas.



Nombrando módulos

- Para nombrar módulos (funciones) se recomienda seguir las siguientes recomendaciones:
 - El nombre debe ser de una sola palabra.
 - El nombre debe ser significativo.
 - Van seguidos de un par de paréntesis.
- **Ejemplo:** Nombres válidos e inválidos para un módulo que calcula el salario bruto de un empleado

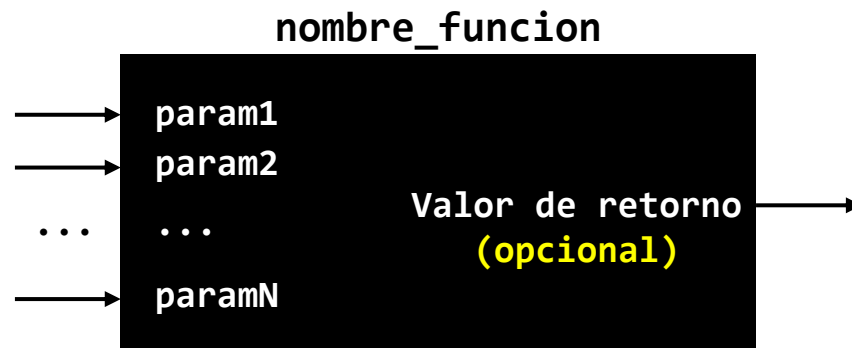
Nombres sugeridos	Comentarios
<code>calcular_salario_bruto()</code>	Bueno
<code>calcular_bruto()</code>	Bueno: la mayoría interpretaría "bruto" como abreviatura de "salario bruto"
<code>cal_sal_br()</code>	Válido , pero críptico
<code>calcular_salario_bruto_para_un_empleado()</code>	Válido , pero torpe
<code>calcular bruto()</code>	No válido: contiene un espacio
<code>calcularsalariobruto()</code>	Válido , pero difícil de leer sin separadores

Funciones en Python - Definición

- **Sintaxis:** Una función se define mediante la palabra clave **def** seguido del nombre del nombre y los parámetros entre paréntesis:

```
def nombre_funcion(param1, param2 ,... , paramN):  
    cuerpo de la funcion
```

- **Donde:**
 - **Nombre de la función:** Identifica la función; con este se llama (invoca).
 - **Parámetros:** Lista de entradas de la función (puede estar vacía).
 - **Cuerpo de la función:** Conjunto de instrucciones que realizan la tarea (van indentadas).
- **Importante:** Definir una función no la ejecuta: solo se ejecuta cuando se la llama.
- **Diagrama de caja negra:** Resume la interfaz de la función (entradas/salidas) sin entrar en los detalles internos.



Tipos de funciones según entradas y salidas

- **Caso 1:** La función solo realiza una acción (mostrar el menú). No recibe ni entrega ningún valor.

```
def mostrar_menu():  
    print("\n--- Áreas de figuras ---")  
    print("1. Círculo")  
    print("2. Rectángulo")  
    print("3. Triángulo")  
    print("4. Salir")
```

mostrar_menu



- **Caso 2:** La función no recibe datos, pero entrega un valor con **return**.

```
def leer_opcion():  
    opcion = input("Opción: ")  
    return opc
```

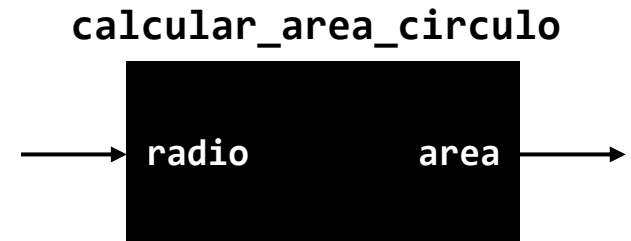
leer_opcion



Tipos de funciones según entradas y salidas

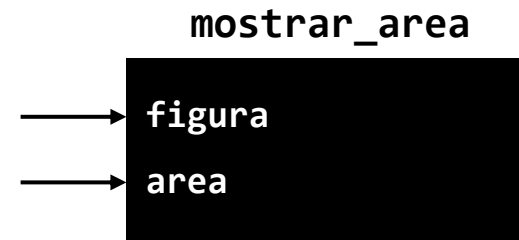
- **Caso 3:** La función recibe datos y entrega un valor con **return**.

```
def calcular_area_circulo(radio):  
    area = PI * radio ** 2  
    return area
```



- **Caso 4:** La función recibe datos pero no entrega ningún valor

```
def mostrar_area(figura, area):  
    print(f"Área del {figura}: {area:.2f}")
```



Funciones en Python - Invocación

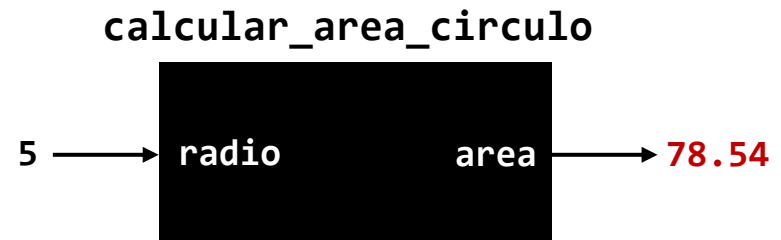
- **Invocar (llamar) una función:** escribir su nombre con los argumentos entre paréntesis

```
[variable =] nombre_funcion(arg1, arg2 ,... , argN)
```

- **Donde:**
 - **Argumentos:** los valores que se envían, y se asocian a los parámetros **por posición**. Pueden ser números, variables o expresiones.
 - Si la función **devuelve** un valor, se guarda en una variable o se usa directamente. Si no devuelve nada, se la llama sola.
- **Ejemplo:** Empleando la función `calcular_area_circulo()`, calcular el área de un círculo de radio 5 y llevar el resultado a una variable llamada área.

```
area = calcular_area_circulo(5)
```

[Link simulación](#)



Trabajando con funciones

Parte 1: Definición de funciones

```
PI = 3.1416

def celsius_to_fahrenheit(c):
    return (9 / 5) * c + 32

def fahrenheit_to_celsius(f):
    return (f - 32) * 5 / 9

def calcular_area_circulo(radio):
    return PI * radio ** 2

def calcular_area_rectangulo(base, altura):
    return base * altura

def mostrar_menu():
    print("--- Áreas de figuras ---")

def mostrar_area(figura, area):
    print(f"Área del {figura}: {area:.2f}")

...
```

	celcius_to_fahrenheit	
--	-----------------------	--

	fahrenheit_to_celcius	
--	-----------------------	--

	calcular_area_circulo	
--	-----------------------	--

	calcular_area_rectangulo	
--	--------------------------	--

	mostrar_menu	
--	--------------	--

	mostrar_area	
--	--------------	--

Trabajando con funciones

Parte 2: Invocación de las funciones.

```
...

# Invocaciones
mostrar_menu()                                # 1. sin argumentos, sin guardar nada

temp_f = celsius_to_fahrenheit(25)           # 2. se guarda el retorno (77.0)
print(celsius_to_fahrenheit(100))            # 3. se usa directamente en print (212.0)
temp_c = fahrenheit_to_celsius(temp_f)        # 4. el argumento es una variable (25.0)

area = calcular_area_circulo(5)               # 5. un argumento (78.54)
mostrar_area("círculo", area)                 # 6. dos argumentos, sin guardar nada
mostrar_area("rectángulo", calcular_area_rectangulo(4, 3)) # 7. una llamada dentro de otra
```

[Link simulación](#)

Resultado

PI	3.1416
temp_f	77.0
temp_c	25.0
area	78.54

```
--- Áreas de figuras ---
212.0
Área del círculo: 78.54
Área del rectángulo: 12.00
```

Probar una función

- **Prueba unitaria:** verificar una función de forma aislada, con varios casos de uso.
- Cada **caso de prueba** compara el resultado **esperado** con el **obtenido**.
- **Integración:** proceso de combinar las funciones ya desarrolladas en el programa completo (menú, lectura de datos y ciclo principal).
- **¿Por qué es importante probar antes de integrar?**
 - Si una función falla de forma aislada, el error se encuentra en ella.
 - Si falla dentro del programa completo, el origen puede estar en la función, en la lectura de datos o en el menú, y localizarlo resulta más costoso.
 - Una función verificada puede reutilizarse con confianza.

Tipo de caso	Qué prueba	Ejemplo
Típico	Uso habitual	<code>calcular_area_rectangulo(4, 3) → 12</code>
Límite	Valores extremos del rango válido	<code>calcular_area_rectangulo(0, 5) → 0</code>
Especial	Valores atípicos o inválidos	<code>calcular_area_rectangulo(-4, 3) → -12</code> , que la función no rechaza

Pruebas con `assert`

- **Sintaxis:** Una verificación se escribe con la palabra clave `assert`, seguida de una condición y, opcionalmente, un mensaje:

```
assert condición [, "Mensaje opcional"]
```

- **Donde:**
 - **Condición:** expresión que debe ser verdadera, normalmente la comparación `llamada == resultado_esperado`.
 - **Mensaje (opcional):** texto que se muestra si la condición es falsa.
- **Si la condición se cumple**, el programa continúa sin mostrar nada.
- **Si no se cumple**, Python lanza `AssertionError` e indica la línea que falló o muestra el mensaje (en caso de que este se haya definido).

```
# Sin mensaje
assert calcular_area_rectangulo(4, 3) == 12

# Con mensaje
assert calcular_area_rectangulo(4, 3) == 12, "El área de 4 x 3 debe ser 12"
```

[Link simulación](#)

Salida en pantalla (Vacía – Todo salió bien)

Pruebas con `assert`: casos y tips

Tips importantes:

1. **Una condición por `assert`:** Si se agrupan varias con `and`, al fallar no se sabe cuál causó el error.
2. **Primero el valor obtenido, después el esperado:** `llamada == esperado`. Mantener el orden en todas las pruebas evita confusiones.
3. **No compare decimales con `==`:** `0.1 + 0.2 == 0.3` da `False`. Para decimales se recomienda usar la función `round`. Por ejemplo: `round(resultado, 2) == esperado`.

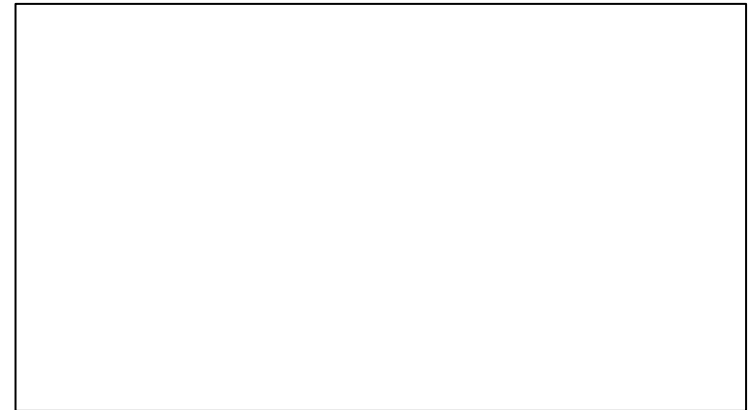
```
# Casos típicos
assert calcular_area_rectangulo(4, 3) == 12
assert calcular_area_rectangulo(2.5, 4) == 10.0

# Casos límite
assert calcular_area_rectangulo(0, 5) == 0
assert calcular_area_rectangulo(1, 1) == 1

# Caso especial
assert calcular_area_rectangulo(-4, 3) == -12
```

[Link simulación](#)

Salida en pantalla (Vacía – Todo salió bien)



Pruebas con `assert`: casos y tips

Tips importantes:

4. **No escriba el `assert` entre paréntesis con coma:** `assert (condicion, "mensaje")` nunca falla, porque Python lo interpreta como una tupla no vacía. Python avisa con un `SyntaxWarning`.
5. **Solo pruebe con funciones que devuelvan un valor:** `mostrar_menu()` y `mostrar_area()` imprimen, y un `assert` no comprueba lo que sale en pantalla.
6. **`assert` es para probar el programa, no para validar lo que escribe el usuario:** Si se ejecuta Python con la opción `-O` (`python -O programa.py`), los `assert` se ignoran. Los datos de entrada se validan con condicionales.

```
# Tip 4: con paréntesis nunca falla
assert (calcular_area_rectangulo(4, 3) == 13, "mensaje")    # no falla

# Sin paréntesis, sí falla
assert calcular_area_rectangulo(4, 3) == 13, "mensaje"      # AssertionError
```

[Link simulación](#)

Salida en pantalla

```
SyntaxWarning: assertion is always true, perhaps remove parentheses?
...
AssertionError: mensaje
```

¿Está bien esta función?

Supóngase que al codificarse la función `calcular_area_rectangulo` quedo escrita de la siguiente manera:

```
def calcular_area_rectangulo(base, altura):  
    area = base + altura  
    return area  
  
assert calcular_area_rectangulo(2, 2) == 4
```

[Link simulación](#)

Preguntas:

- ¿Sale algo?
- ¿Con esto es suficiente para estar tranquilo?

¿Está bien esta función?

Supóngase que al codificarse la función `calcular_area_rectangulo` quedo escrita de la siguiente manera:

```
def calcular_area_rectangulo(base, altura):  
    area = base + altura  
    return area  
  
assert calcular_area_rectangulo(2, 2) == 4  
assert calcular_area_rectangulo(4, 3) == 12, "El área de 4 x 3 debe ser 12"
```

[Link simulación](#)

Salida en pantalla

```
Traceback (most recent call last):  
...  
AssertionError: El área de 4 x 3 debe ser 12
```

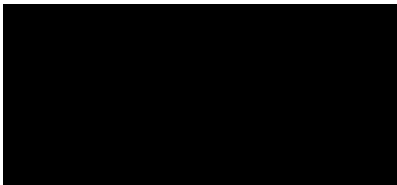
Preguntas:

- ¿Sale algo?, ¿A qué se debe?
- ¿Si hay errores, como se corregirían?

Moraleja: un solo caso puede dar falsa confianza. Varios casos, de distintos tipos, permiten descubrir el error.

Tipos de funciones: Resumen

Tipos de funciones según parámetros y retorno

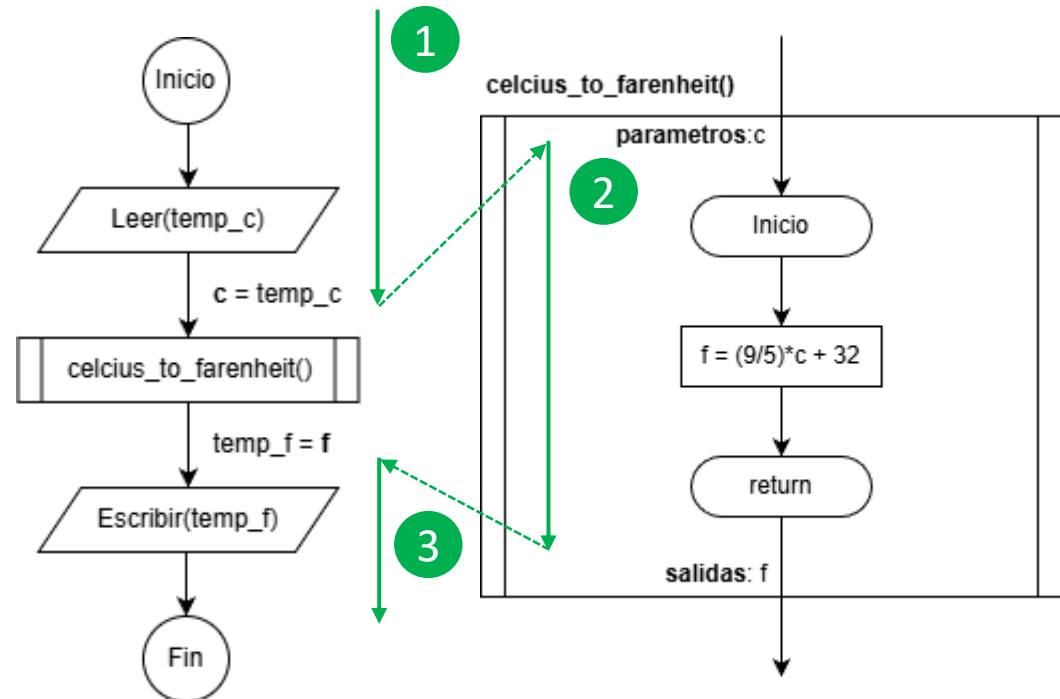
	No devuelve un valor	Devuelve un valor (return)
Sin parámetros	<code>mostrar_menu()</code> <i>Caso 1</i> <code>mostrar_menu</code> 	<code>leer_opcion()</code> <i>Caso 2</i> <code>leer_opcion</code> 
Con parámetros	<code>mostrar_area(figura, area)</code> <i>Caso 4</i> <code>mostrar_area</code> 	<code>calcular_area_circulo(radio)</code> <i>Caso 3</i> <code>calcular_area_circulo</code> 

Función llamadora y función llamada

- **Lamar (invocar) una función:** ejecutarla escribiendo su nombre con los datos que recibe, por ejemplo: `temp_f = celsius_to_fahrenheit(temp_c)`.
- **Función llamadora:** la que usa (llama a) otra función.
- **Función llamada:** la que es usada por otra.
- Flujo de ejecución de un programa con funciones:
 - Al llamar a una función, el control pasa a esta (función llamada)
 - Al terminar (**return**), el control vuelve a quien la llamó (función llamadora).

```
def celsius_to_fahrenheit(c):  
    f = (9/5)*c + 32  
    return f  
  
temp_c = float(input("Ingrese los °C: "))  
temp_f = celsius_to_fahrenheit(temp_c)  
  
print(temp_c,"°C =", temp_f,"°F")
```

[Link simulación](#)



Cohesión funcional: una función, una tarea

- Saber cuándo dividir una función en otras es un arte, pero hay una guía.
- **Cohesión funcional:** las instrucciones de una función contribuyen a **una sola tarea**. Es la buena práctica.
- **Regla práctica:** si para describir lo que hace una función tiene que enlazar varias acciones con "y" (lee **y** calcula **y** muestra), probablemente son varias funciones

Código con baja cohesión

```
# Baja cohesión: lee, calcula y muestra
def promedio_de_notas():
    a = float(input("Nota 1: "))
    b = float(input("Nota 2: "))
    print((a + b) / 2)

# Programa principal
promedio_de_notas()
```

[Link simulación](#)

Código con alta cohesión

```
# Alta cohesión: una tarea por función
def leer_nota(mensaje):
    return float(input(mensaje))

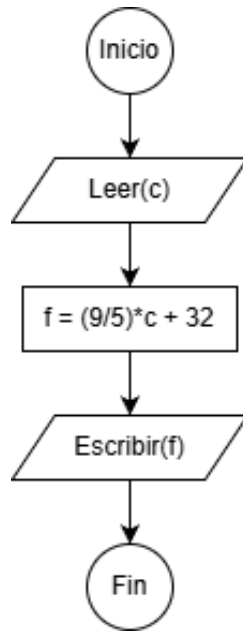
def calcular_promedio(a, b):
    return (a + b) / 2

# Programa principal
nota1 = leer_nota("Nota 1: ")
nota2 = leer_nota("Nota 2: ")
print(calcular_promedio(nota1, nota2))
```

[Link simulación](#)

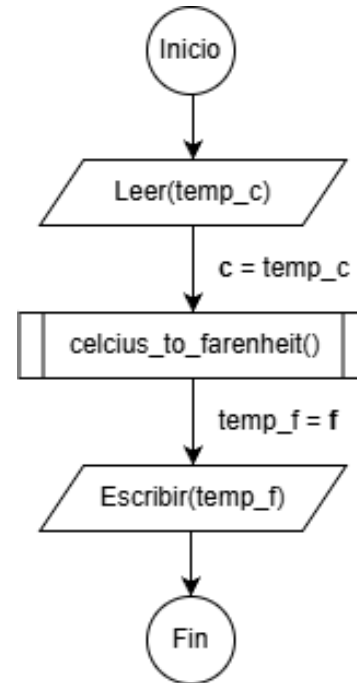
Comparación

- A continuación, se muestra la comparación entre trabajar un programa **sin módulos y con módulos**.



```
temp_c = float(input("Ingrese los °C: "))
temp_f = (9 / 5) * temp_c + 32
print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)



```
def celsius_a_fahrenheit(c):
    f = (9 / 5) * c + 32
    return f

temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)

print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)

Análisis – Ejemplo área de figuras

Análisis ▶ Diseño ▶ Codificación ▶ Pruebas ▶ Integración

Enunciado

Hacer un programa que muestre un menú para calcular el área de diferentes figuras geométricas:

1. Círculo
2. Rectángulo
3. Triángulo
4. Salir

Según la opción elegida, el programa debe:

- **Círculo:** pedir el radio y calcular el área como $\pi \cdot radio^2$ (use $\pi = 3.1416$).
- **Rectángulo:** pedir la base y la altura, y calcular el área como $base \cdot altura$.
- **Triángulo:** pedir la base y la altura, y calcular el área como $(base \cdot altura) / 2$.

Después de cada cálculo, el programa debe mostrar el área con dos decimales y volver a presentar el menú. Si se digita una opción que no está en el menú, debe informarlo y volver a preguntar. El programa termina únicamente cuando se elige la opción 4.

Análisis – Ejemplo área de figuras

Entradas, proceso y salidas

Análisis ► Diseño ► Codificación ► Pruebas ► Integración

- **Punto de partida:** ¿Qué datos entran y qué datos deben salir?

Opción	Entradas	Proceso	Salida
Círculo (1)	<i>radio</i>	$\pi \cdot radio^2$	Área del círculo, con dos decimales
Rectángulo (2)	<i>base, altura</i>	$base \cdot altura$	Área del rectángulo, con dos decimales
Triángulo (3)	<i>base, altura</i>	$\frac{(base \cdot altura)}{2}$	Área del triángulo, con dos decimales
Salir (4)	ninguna	terminar el programa	ninguna
Otra	ninguna	informar el error	"Opción no válida"

- El programa se ejecuta de manera repetitiva de modo que en cada vuelta del ciclo se muestra el menú y se lee la opción, hasta que se elige la 4.

Análisis – Ejemplo área de figuras

Resultado

Análisis ► Diseño ► Codificación ► Pruebas ► Integración

- **Punto de partida:** ¿Qué resultado debe dar el programa para unos datos conocidos?

Opción	Datos	Resultado esperado	Tipo
Círculo (1)	radio = 5	78.54	típico
Círculo (1)	radio = 0	0.00	límite
Rectángulo (2)	base = 4, altura = 3	12.00	típico
Rectángulo (2)	base = 0, altura = 5	0.00	límite
Triángulo (3)	base = 6, altura = 2	6.00	típico
Triángulo (3)	base = 0, altura = 5	0.00	límite
7	—	"Opción no válida" y se vuelve a mostrar el menú	especial

- Esta tabla se convierte después en los assert en la etapa de pruebas.

Diseño – Ejemplo área de figuras

De tareas a funciones

Análisis ▶ **Diseño** ▶ Codificación ▶ Pruebas ▶ Integración

- **Punto de partida:** ¿Qué tareas hay en el programa y cómo las dividimos en funciones?
- **Criterio:** Una función, una tarea (cohesión funcional)

Tarea	Función	Parámetros	Retorno	Caso
Mostrar el menú	mostrar_menu	ninguno	nada	1
Leer la opción	leer_opcion	ninguno	opcion	2
Calcular el área del círculo	calcular_area_circulo	radio	area	3
Calcular el área del rectángulo	calcular_area_rectangulo	base, altura	area	3
Calcular el área del triángulo	calcular_area_triangulo	base, altura	area	3
Mostrar el área	mostrar_area	figura, area	nada	4

Codificación – Ejemplo area de figuras

Análisis ▶ Diseño ▶ **Codificación** ▶ Pruebas ▶ Integración

Funciones

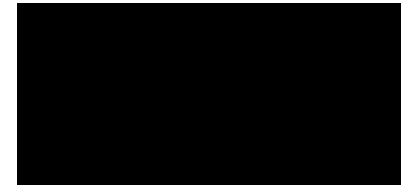
```
def mostrar_menu():  
    print("\n--- Áreas de figuras ---")  
    print("1. Círculo")  
    print("2. Rectángulo")  
    print("3. Triángulo")  
    print("4. Salir")
```

```
def leer_opcion():  
    opcion = input("Opción: ")  
    return opcion
```

```
PI = 3.1416
```

```
def calcular_area_circulo(radio):  
    area = PI * radio ** 2  
    return area
```

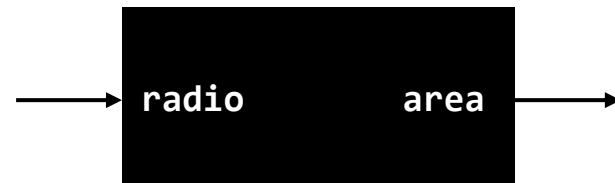
mostrar_menu



leer_opcion



calcular_area_circulo



Codificación – Ejemplo area de figuras

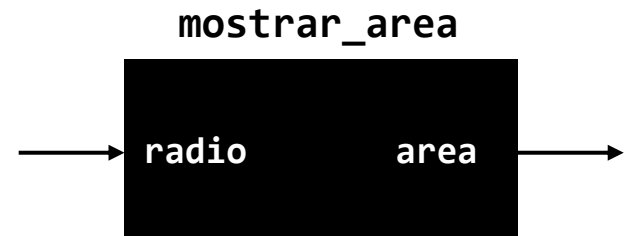
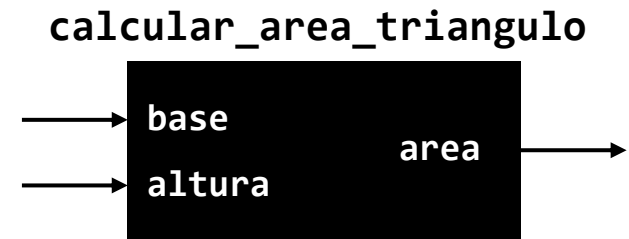
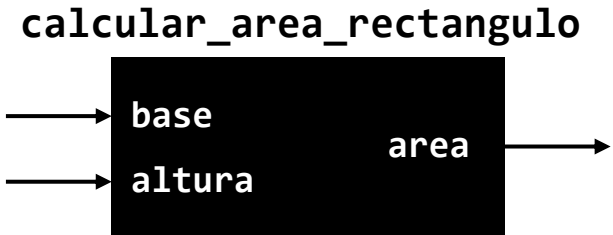
Análisis ▶ Diseño ▶ **Codificación** ▶ Pruebas ▶ Integración

Funciones

```
def calcular_area_rectangulo(base, altura):  
    area = base * altura  
    return area
```

```
def calcular_area_triangulo(base, altura):  
    area = base * altura / 2  
    return area
```

```
def mostrar_area(figura, area):  
    print(f"Área del {figura}: {area:.2f}")
```



Codificación – Ejemplo area de figuras

Análisis ▶ Diseño ▶ **Codificación** ▶ Pruebas ▶ Integración

Archivo: areas.py

- **Paso 1:** Reunimos todas las funciones en un solo archivo

```
# ----- Constantes -----
PI = 3.1416

# ----- Funciones -----
def mostrar_menu():
    print("\n--- Áreas de figuras ---")
    print("1. Círculo")
    print("2. Rectángulo")
    print("3. Triángulo")
    print("4. Salir")

def leer_opcion():
    opcion = input("Opción: ")
    return opcion

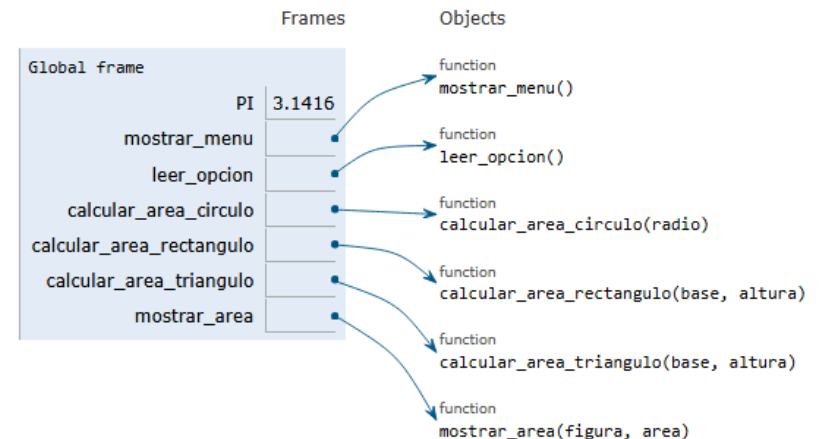
def calcular_area_circulo(radio):
    area = PI * radio ** 2
    return area

def calcular_area_rectangulo(base, altura):
    area = base * altura
    return area
```

```
def calcular_area_triangulo(base, altura):
    area = base * altura / 2
    return area

def mostrar_area(figura, area):
    print(f"Área del {figura}: {area:.2f}")
```

[Link simulación](#)



Pruebas – Ejemplo area de figuras

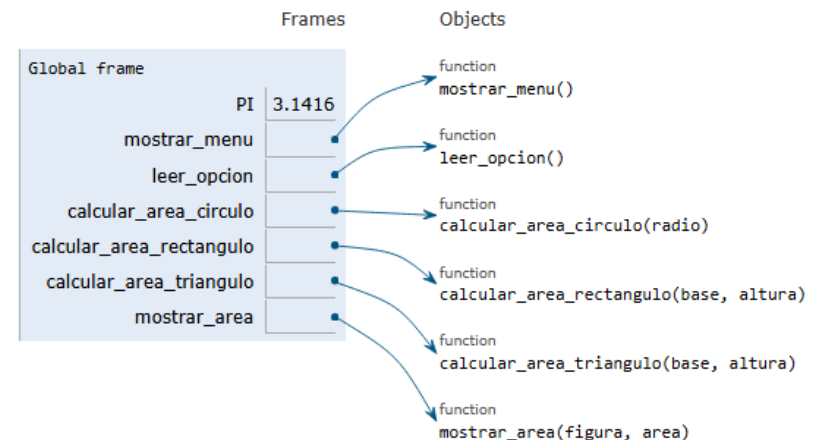
Análisis ▶ Diseño ▶ Codificación ▶ **Pruebas** ▶ Integración

Archivo: areas.py

- **Paso 2:** Agregamos las pruebas de las funciones de cálculo.

```
# ----- Constantes -----  
# ----- Funciones -----  
# ... (sin cambios)  
  
# ----- Pruebas -----  
# Círculo  
assert round(calcular_area_circulo(5), 2) == 78.54  
assert calcular_area_circulo(0) == 0  
  
# Rectángulo  
assert calcular_area_rectangulo(4, 3) == 12  
assert calcular_area_rectangulo(0, 5) == 0  
  
# Triángulo  
assert calcular_area_triangulo(6, 2) == 6  
assert calcular_area_triangulo(0, 5) == 0
```

[Link simulación](#)



Integración – Ejemplo area de figuras

Análisis ▶ Diseño ▶ Codificación ▶ Pruebas ▶ **Integración**

Archivo: areas.py

- **Paso 3:** Integramos las funciones en el programa principal.

```
# ... (sin cambios)

# ----- Programa principal -----
opcion = ""
while opcion != "4":
    mostrar_menu()
    opcion = leer_opcion()

    if opcion == "1":
        radio = float(input("Radio: "))
        area = calcular_area_circulo(radio)
        mostrar_area("círculo", area)
    elif opcion == "2":
        base = float(input("Base: "))
        altura = float(input("Altura: "))
        area = calcular_area_rectangulo(base, altura)
        mostrar_area("rectángulo", area)
    elif opcion == "3":
        base = float(input("Base: "))
        altura = float(input("Altura: "))
        area = calcular_area_triangulo(base, altura)
        mostrar_area("triángulo", area)
    elif opcion != "4":
        print("Opción no válida")
```

Print output

```
--- Áreas de figuras ---
1. Círculo
2. Rectángulo
3. Triángulo
4. Salir
Opción: 3
Base: 4
Altura: 5
Área del triángulo: 10.00

--- Áreas de figuras ---
1. Círculo
2. Rectángulo
3. Triángulo
4. Salir
Opción: 1
Radio: 3
Área del círculo: 28.27

--- Áreas de figuras ---
1. Círculo
2. Rectángulo
3. Triángulo
4. Salir
Opción: 7
Opción no válida

--- Áreas de figuras ---
1. Círculo
2. Rectángulo
3. Triángulo
4. Salir
Opción: 4
```

Análisis – Ejemplo calificaciones

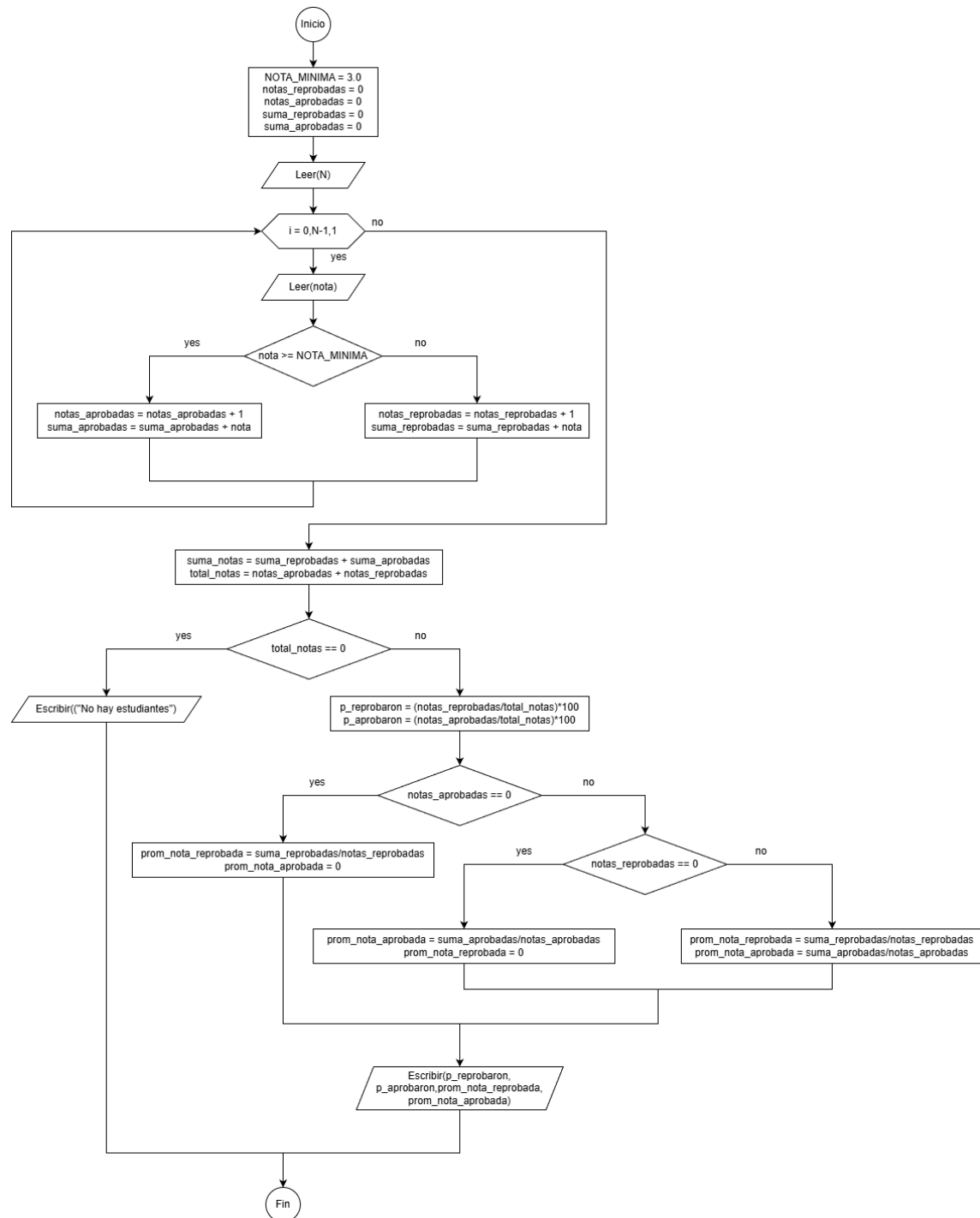
Hacer un programa que lea primero el número de estudiantes de un curso (**N**), y luego lea las **N** notas correspondientes (valores decimales entre 0.0 y 5.0), una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es aprobada si es mayor o igual a 3.0, y reprobada si es menor a 3.0.

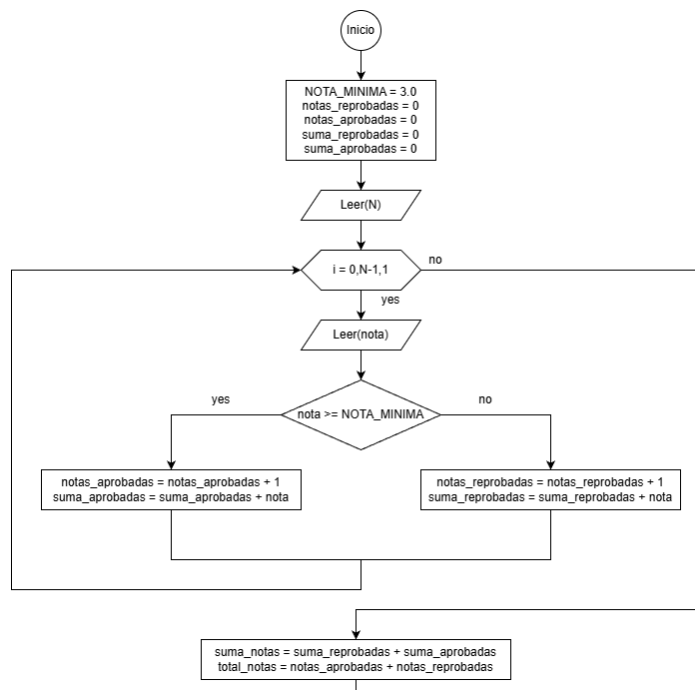
Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.

Ejemplo Calificaciones (No modular)

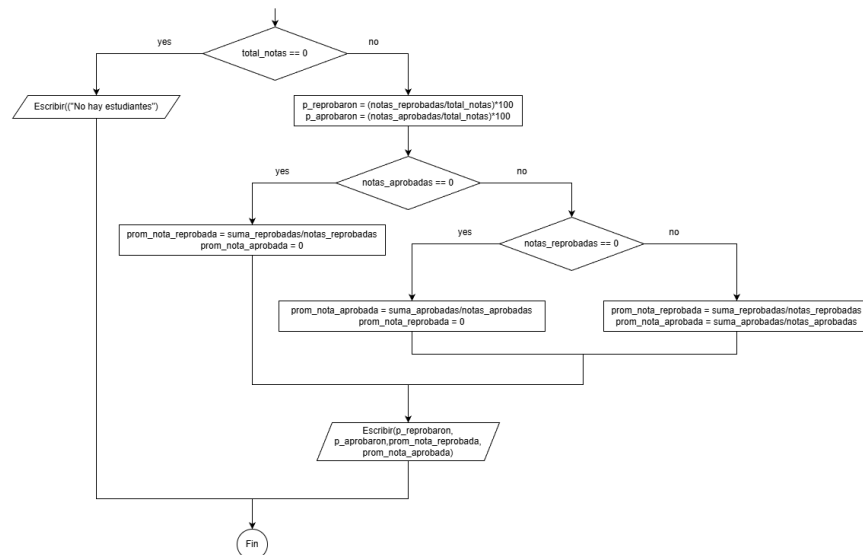


Ejemplo calificaciones



```

NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
...
  
```



```

...
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
        prom_nota_reprobada = 0
    else:
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_nota_reprobada}")
    print(f"Promedio aprobados: {prom_nota_aprobada}")
    print(f"Promedio de notas: {prom}")
  
```

Ejemplo calificaciones

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corrijalo.

```
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
        prom_nota_reprobada = 0
    else:
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_nota_reprobada}")
    print(f"Promedio aprobados: {prom_nota_aprobada}")
    print(f"Promedio de notas: {prom}")
```

Ejemplo calificaciones

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Resultado de ejecución

```
Ingrese el número de estudiantes: 5
Ingrese la nota del estudiante 1: 4
Ingrese la nota del estudiante 2: 2.5
Ingrese la nota del estudiante 3: 3.0
Ingrese la nota del estudiante 4: 1.5
Ingrese la nota del estudiante 5: 4.8
Porcentaje de estudiantes que aprobaron: 60.0%
Porcentaje de estudiantes que reprobaron: 40.0%
Promedio de notas reprobadas: 2.0
Promedio de notas aprobadas: 3.9333333333333336
Promedio general de notas: 3.16
```

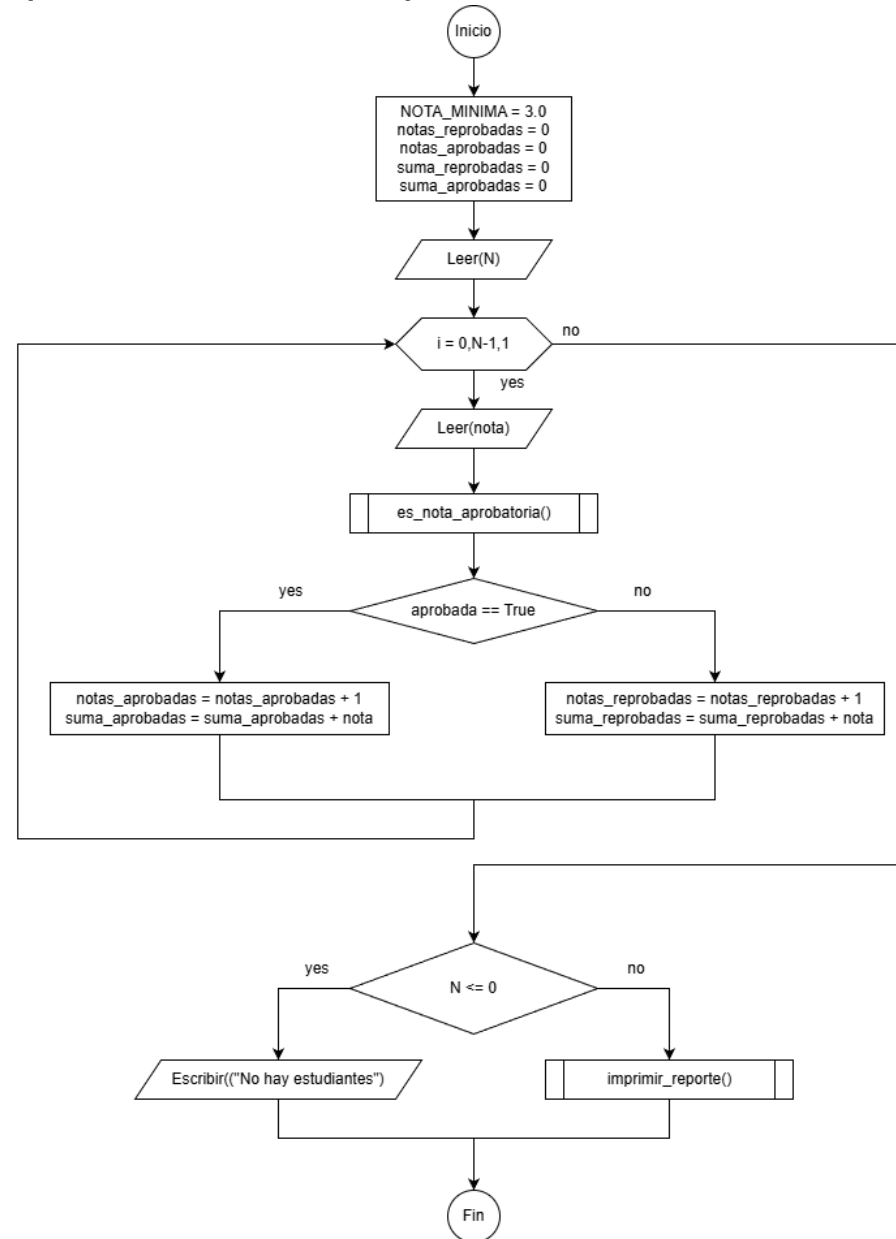
Ejemplo Calificaciones (Modular)

Programa principal

```
# Programa principal
est_aprob = 0
est_reprob = 0
suma_aprob = 0
suma_reprob = 0

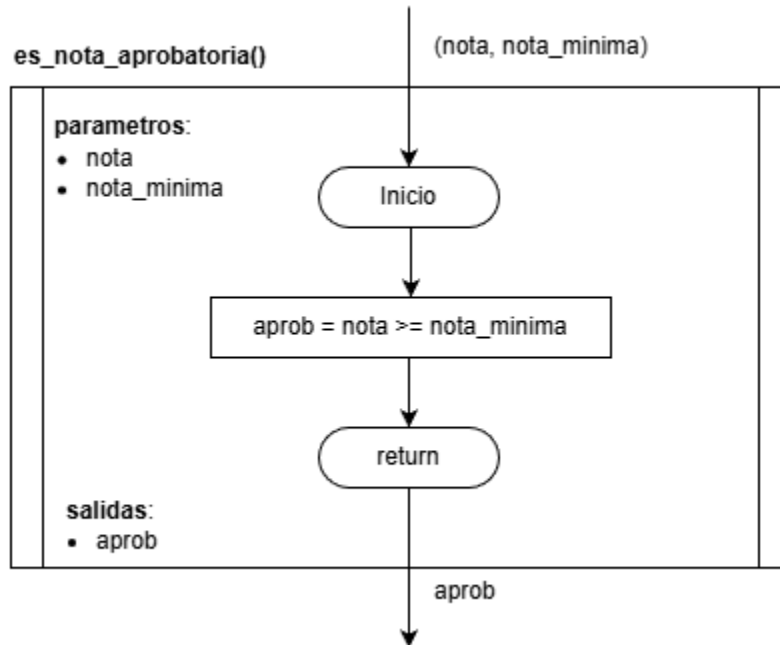
N = int(input("Número de estudiantes: "))
for i in range(N):
    nota = float(input("- Digite la nota: "))
    if es_nota_aprobatoria(nota, NOTA_MINIMA):
        est_aprob += 1
        suma_aprob += nota
    else:
        est_reprob += 1
        suma_reprob += nota

if N <= 0:
    print("No hay estudiantes")
else:
    imprimir_reporte(est_aprob,
                    est_reprob,
                    suma_aprob,
                    suma_reprob)
```



Ejemplo Calificaciones (Modular)

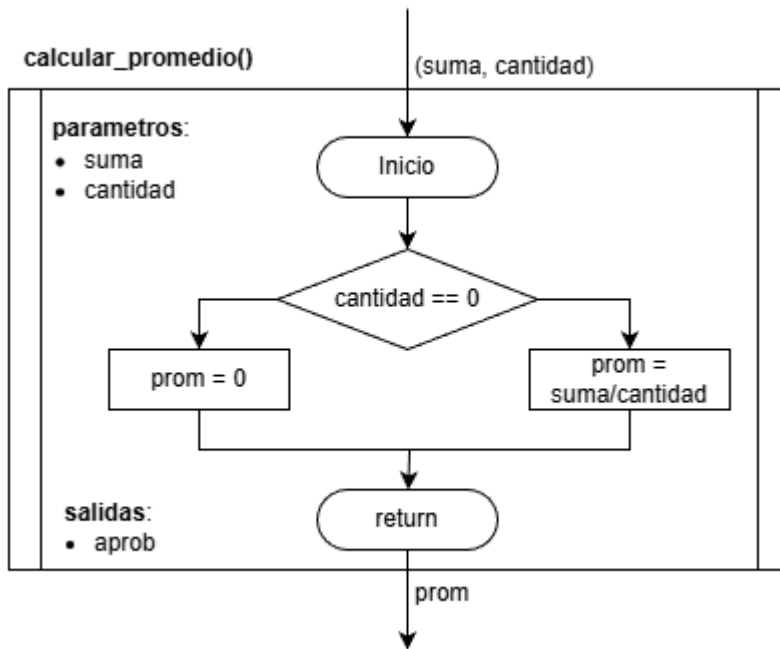
Modulo `es_nota_aprobatoria()`



```
def es_nota_aprobatoria(nota, nota_minima):  
    aprob = nota >= nota_minima  
    return aprob
```

Ejemplo Calificaciones (Modular)

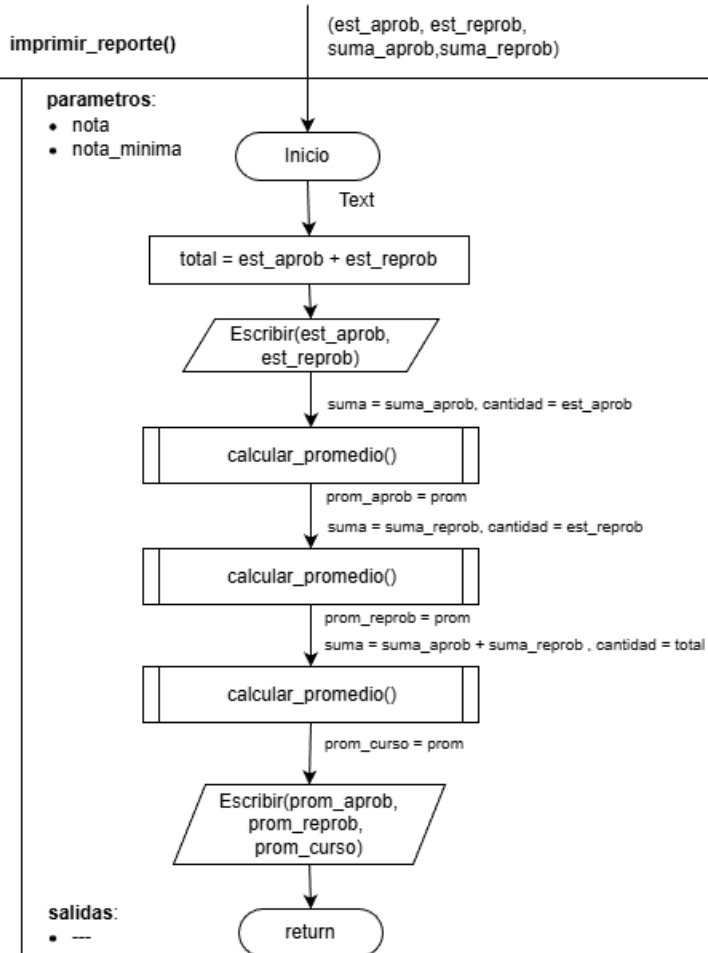
Modulo `calcular_promedio()`



```
def calcular_promedio(suma, cantidad):  
    if cantidad == 0:  
        prom = 0  
    else:  
        prom = suma/cantidad  
    return prom
```

Ejemplo Calificaciones (Modular)

Modulo `imprimir_reporte()`



```
def imprimir_reporte(est_aprob, est_reprob,
                    suma_aprob, suma_reprob):
    total = est_aprob + est_reprob
    print(f"Ganaron: {est_aprob}")
    print(f"Perdieron: {est_reprob}")
    prom_abrob = calcular_promedio(suma_aprob,
                                   est_aprob)
    prom_reprob = calcular_promedio(suma_reprob,
                                    est_reprob)
    prom_curso = calcular_promedio(suma_aprob + suma_reprob,
                                   total)
    print(f"Prom aprob: {prom_aprob:.2f}")
    print(f"Prom reprob: {prom_reprob:.2f}")
    print(f"Prom nota: {prom_curso:.2f}")
```

Ejemplo Calificaciones (Modular)

Codigo completo

```
# Constantes
NOTA_MINIMA = 3.0

# ----- Funciones -----
def es_nota_aprobatoria(nota, nota_minima):
    aprob = nota >= nota_minima
    return aprob

def calcular_promedio(suma, cantidad):
    if cantidad == 0:
        prom = 0
    else:
        prom = suma/cantidad
    return prom

def imprimir_reporte(est_aprob, est_reprob,
                    suma_aprob, suma_reprob):
    total = est_aprob + est_reprob
    print(f"Ganaron: {est_aprob}")
    print(f"Perdieron: {est_reprob}")
    prom_aprob = calcular_promedio(suma_aprob,
                                   est_aprob)
    prom_reprob = calcular_promedio(suma_reprob,
                                    est_reprob)
    prom_curso = calcular_promedio(suma_aprob +
                                   suma_reprob,
                                   total)

    print(f"Prom aprob: {prom_aprob:.2f}")
    print(f"Prom reprob: {prom_reprob:.2f}")
    print(f"Prom nota: {prom_curso:.2f}")
```

```
# ----- Programa principal -----
est_aprob = 0
est_reprob = 0
suma_aprob = 0
suma_reprob = 0

N = int(input("Número de estudiantes: "))
for i in range(N):
    nota = float(input("- Digite la nota: "))
    if es_nota_aprobatoria(nota, NOTA_MINIMA):
        est_aprob += 1
        suma_aprob += nota
    else:
        est_reprob += 1
        suma_reprob += nota

if N <= 0:
    print("No hay estudiantes")
else:
    imprimir_reporte(est_aprob,
                    est_reprob,
                    suma_aprob,
                    suma_reprob)
```

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/defining-your-own-python-function/>
8. <https://realpython.com/python-built-in-functions/>
9. <https://realpython.com/ref/best-practices/functions/>
10. <https://makecode.microbit.org/courses/csintro>
11. <https://makecode.microbit.org/courses>
12. <https://realpython.com/python-while-loop/>
13. <https://realpython.com/ref/keywords/while/>
14. <https://cs.stanford.edu/people/nick/py/>

Eso es todo amigos

